Iris Classification with k-Nearest Neighbors

Author: Manuel Eugenio Morocho Cayamcela, PhD

Introduction

Use supervised learning to classify 150 Iris flowers into three species from four measurements: sepal length, sepal width, petal length, and petal width (in centimeters). This lesson covers loading data, exploration, preprocessing, training a k-nearest neighbors (k-NN) classifier, saving the model, and evaluating predictions.



Iris setosa



Iris versicolor



Iris virgínica

Setup

If needed, install the packages once with `%pip install pandas matplotlib seaborn scikit-learn joblib`. Restart the kernel after installation if requested.

In [ ]:
```
from pathlib import Path

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, ConfusionMatrixD
```

Load the Iris dataset

Read the Iris CSV directly from GitHub using pandas. An internet connection is required.

In [ ]:
```
github_url = "https://raw.githubusercontent.com/eugeniomorocho/ArtificialIntelligen
iris = pd.read_csv(github_url)
```

Explore and prepare the data

Run each cell separately and inspect its output before continuing.

Preview the first five records.

In [ ]:
```
iris.head()
```

Inspect column types and non-null counts to identify missing values.

In [ ]:
```
iris.info()
```

Summarize the numeric columns.

In [ ]:
```
iris.describe()
```

Count the examples of each species.

In [ ]:
```
iris["Species"].value_counts()
```

Remove `Id` : it identifies records rather than measuring flowers.

In [ ]:
```
iris = iris.drop(columns="Id")
```

Visualize the measurements

Compare sepal and petal measurements by species. Setosa is particularly distinct in petal measurements; versicolor and virginica overlap more. These plots help assess whether the measurements can separate the classes.

Sepal measurements

In [ ]:
```
sns.scatterplot(data=iris, x="SepalLengthCm", y="SepalWidthCm", hue="Species")
plt.show()
```

Petal measurements

In [ ]:
```
sns.scatterplot(data=iris, x="PetalLengthCm", y="PetalWidthCm", hue="Species")
plt.show()
```

The pair plot compares every pair of measurements. Histograms on the diagonal show each feature's distribution.

In [ ]:
```
sns.pairplot(iris, hue="Species", diag_kind="hist")
plt.show()
```

Prepare training and test sets

Separate the four predictors (`X`) from the species label (`y`). Reserve 20% of the data for testing. Stratification preserves class proportions, and a fixed random seed makes the split reproducible. Keep the test set out of model fitting.

In [ ]:
```
# Isolate features
X = iris.drop(columns="Species")

# Isolate target variable
y = iris["Species"]

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

Train the k-NN classifier

With `n_neighbors=5` , the classifier predicts a flower's species from the majority class among its five nearest training examples. Fit the model on the training set only.

In [ ]:
```
from sklearn.neighbors import KNeighborsClassifier

classifier = KNeighborsClassifier(n_neighbors=5)
classifier.fit(X_train, y_train)
```

Evaluate on unseen data

Predict the species of the held-out examples.

In [ ]:
```
predictions = classifier.predict(X_test)
```

Accuracy

Accuracy is the fraction of correctly classified test examples.

In [ ]:
```
print(f"Test accuracy: {accuracy_score(y_test, predictions):.3f}")
```

Classification report

The report summarizes each species:

- **Precision:** TP / (TP + FP), the fraction of predictions for that species that are correct.
- **Recall:** TP / (TP + FN), the fraction of actual flowers of that species correctly identified.
- **F1-score:** the harmonic mean of precision and recall.
- **Support:** the number of true examples of that species in the test set.

Interpret the results generated below; scores can change with the split and model settings.

In [ ]:
```
print(classification_report(y_test, predictions, zero_division=0))
```

Confusion matrix

Rows show actual classes and columns show predicted classes. Diagonal entries are correct predictions.

In [ ]:
```
ConfusionMatrixDisplay.from_predictions(
    y_test, predictions, labels=classifier.classes_, cmap="Blues"
)
plt.title("Iris classification: confusion matrix")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

Save and reload the trained model

Save the fitted classifier for reuse without retraining. Only load model files you trust. The file is written to the current working directory.

In [ ]:
```
import joblib

model_path = Path("knn_model.pkl")
joblib.dump(classifier, model_path)
```

In [ ]:
```
loaded_classifier = joblib.load(model_path)
loaded_classifier.predict(X_test.head())
```

Activity: Data Preprocessing and Customer Classification with k-NN

Objective

Explore a classification dataset, train k-NN, and select a suitable value of `k`. Work individually or in a group, share the work fairly, and justify your decisions.

Where to find datasets

For this short activity, choose a small tabular dataset with a clearly documented categorical target. Prefer CSV files and numeric predictors so that data preparation remains manageable.

- UCI Machine Learning Repository: a useful starting point for classification datasets.
- Kaggle Datasets: browse datasets and select one with a documented target and downloadable tabular files. Check access requirements before choosing it.
- Awesome Public Datasets on GitHub: a topic-based directory linking to datasets; check the original source for files and documentation.
- Ecuador Open Data Portal — CSV datasets: explore Ecuadorian public data available in CSV format.
- INEC Open Data Bank: find Ecuadorian statistical information and consult the accompanying documentation.
- U.S. Government Open Data: search public datasets by topic or organization.

Government portals are useful for local or applied topics, but their tables may require more cleaning and may not have a ready-made classification target. Verify the target column and download format before committing to a dataset.

Instructions

1. **Find and explore a dataset (10 minutes).** Load a classification dataset, inspect its structure, and clean or transform it as needed.

In [1]:
```
# Import the dataset
from pathlib import Path
import glob, os, re, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
warnings.filterwarnings("ignore")

from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, FunctionTransformer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.dummy import DummyClassifier
from sklearn.metrics import (accuracy_score, balanced_accuracy_score, f1_score, cla
                             ConfusionMatrixDisplay)

# ---------------- PARÁMETROS DEL EXPERIMENTO (se pueden ajustar) ----------------
RANDOM_STATE = 42
SAMPLE_N = 20_000          # el archivo completo tiene cientos de miles de filas: k
MIN_CLASS_SHARE = 0.005    # clases con menos del 0,5 % de las filas se descartan (
TARGET_CANDIDATES = ["property_type_en", "property_type"]   # columna objetivo (se
```

In [2]:
```
import pandas as pd

ruta = r"C:\Users\Usuario\Documents\2026\BRIGITH\MAESTRIA\FUNDAMENTOS DE INTELIGENC
df = pd.read_csv(ruta)

df.head()
```

Out[2]:

	transaction_id	instance_date	reg_type_label	area_id	area_name_en	project_name	mas
0	1-11-1975-2	1975-01-01	Ready	232.0	Mirdif	NaN	
1	1-11-1990-1	1990-02-12	Ready	268.0	Al Musalla (Dubai)	NaN	
2	1-11-1992-1	1992-04-20	Ready	270.0	Al Safaa	NaN	
3	1-11-1993-1	1993-01-27	Ready	231.0	Al Mamzer	NaN	
4	1-11-1997-62	1997-09-08	Ready	271.0	Al Karama	NaN	

In [5]:
```
info = df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1372277 entries, 0 to 1372276
Data columns (total 20 columns):
 #   Column               Non-Null Count    Dtype
---  ------               --------------    -----
 0   transaction_id       1372277 non-null  str
 1   instance_date        1372277 non-null  str
 2   reg_type_label       1372277 non-null  str
 3   area_id              1372149 non-null  float64
 4   area_name_en         1372149 non-null  str
 5   project_name         1088508 non-null  str
 6   master_project_name  1129462 non-null  str
 7   building_name        973945 non-null   str
 8   property_type_label  1372277 non-null  str
 9   property_sub_type    1174322 non-null  str
 10  usage_label          1372277 non-null  str
 11  rooms_bucket         1092623 non-null  float64
 12  has_parking          1333380 non-null  object
 13  area_sqm             1372276 non-null  float64
 14  price_aed            1372277 non-null  int64
 15  price_psf            1372093 non-null  float64
 16  procedure_id         1348513 non-null  float64
 17  procedure_name_en    1348513 non-null  str
 18  is_pre_registration  1348513 non-null  object
 19  quality_flags        1372277 non-null  int64
dtypes: float64(5), int64(2), object(2), str(11)
memory usage: 209.4+ MB
```

In [9]:
```
from pathlib import Path
import pandas as pd

ruta = Path(
    r"C:\Users\Usuario\Documents\2026\BRIGITH\MAESTRIA"
```

```
    r"\FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL\SEMANA 2"
    r"\ACTIVIDAD 5\dld_sales_transactions.csv"
)

df = pd.read_csv(ruta, low_memory=False)
filas_iniciales = len(df)

# Quitar espacios y convertir textos vacíos o marcadores comunes en faltantes.
def limpiar_texto(valor):
    if isinstance(valor, str):
        valor = valor.strip()
        if valor.lower() in {"", "nan", "none", "null", "n/a"}:
            return pd.NA
    return valor

for columna in df.select_dtypes(include="object").columns:
    df[columna] = df[columna].map(limpiar_texto)

# Normalizar valores booleanos y conservar los faltantes.
def normalizar_booleano(valor):
    if pd.isna(valor):
        return pd.NA

    if isinstance(valor, str):
        valor = valor.strip().lower()

    if valor in (True, 1, "true", "1", "yes", "si", "sí"):
        return True
    if valor in (False, 0, "false", "0", "no"):
        return False

    raise ValueError(f"Valor booleano inesperado: {valor!r}")

for columna in ["has_parking", "is_pre_registration"]:
    df[columna] = df[columna].map(normalizar_booleano).astype("boolean")

# Convertir fechas y asegurar que las columnas numéricas tengan el tipo adecuado.
df["instance_date"] = pd.to_datetime(df["instance_date"], errors="coerce")

columnas_numericas = [
    "area_id", "rooms_bucket", "area_sqm", "price_aed",
    "price_psf", "procedure_id", "quality_flags"
]
for columna in columnas_numericas:
    df[columna] = pd.to_numeric(df[columna], errors="coerce")

# Un precio por pie cuadrado de cero no es válido; se marca como faltante.
df.loc[df["price_psf"].le(0), "price_psf"] = pd.NA

# Revisar el resultado y guardar una copia limpia.
print(f"Filas conservadas: {len(df):,} de {filas_iniciales:,}")
print(f"Fechas faltantes o inválidas: {df['instance_date'].isna().sum():,}")
print(f"Duplicados exactos: {df.duplicated().sum():,}")
print("\nFaltantes por columna:")
print(df.isna().sum().sort_values(ascending=False).head(10))
```

```
salida = ruta.with_name("dld_sales_transactions_clean.csv")
df.to_csv(salida, index=False, date_format="%Y-%m-%d")
print(f"\nArchivo limpio guardado en: {salida}")
```

```
Filas conservadas: 1,372,277 de 1,372,277
Fechas faltantes o inválidas: 0
Duplicados exactos: 0

Faltantes por columna:
building_name          398332
project_name           283769
rooms_bucket           279654
master_project_name    242815
property_sub_type      197955
has_parking             38897
is_pre_registration     23764
procedure_name_en       23764
procedure_id            23764
price_psf                 208
dtype: int64

Archivo limpio guardado en: C:\Users\Usuario\Documents\2026\BRIGITH\MAESTRIA\FUNDAME
NTOS DE INTELIGENCIA ARTIFICIAL\SEMANA 2\ACTIVIDAD 5\dld_sales_transactions_clean.cs
v
```

2. **Classify with k-NN (20 minutes).** Split the data into training and test sets, train the classifier, and report accuracy. Select `k` using only training data (for example, cross-validation), then evaluate the chosen model on the test set. Predict a new example outside both sets. Complete the random forest comparison in the companion notebook.

In [12]:
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Cargar los datos
iris = load_iris()
X, y = iris.data, iris.target

# Separar entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    stratify=y,
    random_state=42
)

# Escalado y k-NN en un pipeline para evitar fuga de información
pipeline_knn = Pipeline([
    ("escalador", StandardScaler()),
    ("knn", KNeighborsClassifier())
])
```

```python
# Seleccionar k usando solo entrenamiento y validación cruzada
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

busqueda_knn = GridSearchCV(
    estimator=pipeline_knn,
    param_grid={"knn__n_neighbors": range(1, 21)},
    cv=cv,
    scoring="accuracy",
    n_jobs=-1
)

busqueda_knn.fit(X_train, y_train)

# Evaluar una sola vez con el conjunto de prueba
mejor_knn = busqueda_knn.best_estimator_
predicciones = mejor_knn.predict(X_test)

print("Mejor k:", busqueda_knn.best_params_["knn__n_neighbors"])
print("Exactitud promedio en validación cruzada:",
      round(busqueda_knn.best_score_, 4))
print("Exactitud en prueba:",
      round(accuracy_score(y_test, predicciones), 4))
print("\nMatriz de confusión:")
print(confusion_matrix(y_test, predicciones))
print("\nReporte de clasificación:")
print(classification_report(y_test, predicciones, target_names=iris.target_names))

# Ejemplo nuevo, independiente de los conjuntos de entrenamiento y prueba.
# Orden: largo sépalo, ancho sépalo, largo pétalo, ancho pétalo (cm).
nuevo_ejemplo = [[5.05, 3.45, 1.45, 0.18]]
prediccion = mejor_knn.predict(nuevo_ejemplo)[0]

print("Predicción para el nuevo ejemplo:", iris.target_names[prediccion])
```

```
Mejor k: 3
Exactitud promedio en validación cruzada: 0.9667
Exactitud en prueba: 0.9333

Matriz de confusión:
[[10  0  0]
 [ 0 10  0]
 [ 0  2  8]]

Reporte de clasificación:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.83      1.00      0.91        10
   virginica       1.00      0.80      0.89        10

    accuracy                           0.93        30
   macro avg       0.94      0.93      0.93        30
weighted avg       0.94      0.93      0.93        30

Predicción para el nuevo ejemplo: setosa
```

2. Comparación con Random Forest

Pega este bloque en el notebook compañero. Repite la misma separación para que ambos modelos se evalúen con el mismo conjunto de prueba.

In [13]:
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    stratify=y,
    random_state=42
)

modelo_rf = RandomForestClassifier(
    n_estimators=300,
    random_state=42
)

modelo_rf.fit(X_train, y_train)
predicciones_rf = modelo_rf.predict(X_test)

print("Exactitud de Random Forest:",
      round(accuracy_score(y_test, predicciones_rf), 4))
print("\nReporte de clasificación:")
print(classification_report(
    y_test,
    predicciones_rf,
    target_names=iris.target_names
))
```

```
Exactitud de Random Forest: 0.9

Reporte de clasificación:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.82      0.90      0.86        10
   virginica       0.89      0.80      0.84        10

    accuracy                           0.90        30
   macro avg       0.90      0.90      0.90        30
weighted avg       0.90      0.90      0.90        30
```

3. Selección de *k* usando **solo** datos de entrenamiento (40 %)

Qué hace k y por qué importa

En k-NN, *k* es el número de vecinos que votan. Es el control del compromiso **sesgo–varianza**:

- **k pequeño (p. ej. 1):** la frontera sigue cada punto → memoriza el ruido (**sobreajuste**, alta varianza).
- **k grande:** la frontera se suaviza demasiado y la clase mayoritaria arrasa (**subajuste**, alto sesgo; especialmente grave con clases desbalanceadas).

Método: validación cruzada estratificada de 5 folds en train

Para cada *k* candidato se entrena y valida 5 veces con partes distintas de **train**; se promedia el **F1 macro** (no el accuracy, por el desbalance). Se prueban además dos opciones de votación (`uniform` y `distance` : los vecinos cercanos pesan más) y dos métricas de distancia (euclídea y Manhattan).

In [15]:
```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

# X_train e y_train deben venir de la separación de datos Iris anterior.
pipeline_knn = Pipeline([
    ("escalador", StandardScaler()),
    ("knn", KNeighborsClassifier())
])

param_grid = {
    "knn__n_neighbors": [1, 3, 5, 7, 9, 11, 15, 21, 31, 41, 51, 71],
    "knn__weights": ["uniform", "distance"],
    "knn__p": [1, 2]
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

grid = GridSearchCV(
    estimator=pipeline_knn,
    param_grid=param_grid,
    cv=cv,
    scoring="f1_macro",
    n_jobs=-1,
    return_train_score=True
)

grid.fit(X_train, y_train)
```

```
print("Mejores hiperparámetros:", grid.best_params_)
print(f"F1 macro promedio en validación cruzada: {grid.best_score_:.4f}")
```

```
Mejores hiperparámetros: {'knn__n_neighbors': 3, 'knn__p': 2, 'knn__weights': 'unifo
rm'}
F1 macro promedio en validación cruzada: 0.9665
```

In [16]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

res = pd.DataFrame(grid.cv_results_)

# Filtrar por los mejores pesos y la mejor métrica de distancia.
best_weights = grid.best_params_["knn__weights"]
best_p = grid.best_params_["knn__p"]

curve = res[
    (res["param_knn__weights"] == best_weights)
    & (res["param_knn__p"] == best_p)
].sort_values("param_knn__n_neighbors")

ks = curve["param_knn__n_neighbors"].astype(int)
best_k = int(grid.best_params_["knn__n_neighbors"])

best_row = curve[
    curve["param_knn__n_neighbors"].astype(int) == best_k
].iloc[0]

best_mean = best_row["mean_test_score"]
best_std = best_row["std_test_score"]

# Regla de un error estándar: elegir el k mayor dentro del margen.
standard_error = best_std / np.sqrt(cv.get_n_splits())
eligible_ks = curve.loc[
    curve["mean_test_score"] >= best_mean - standard_error,
    "param_knn__n_neighbors"
].astype(int)

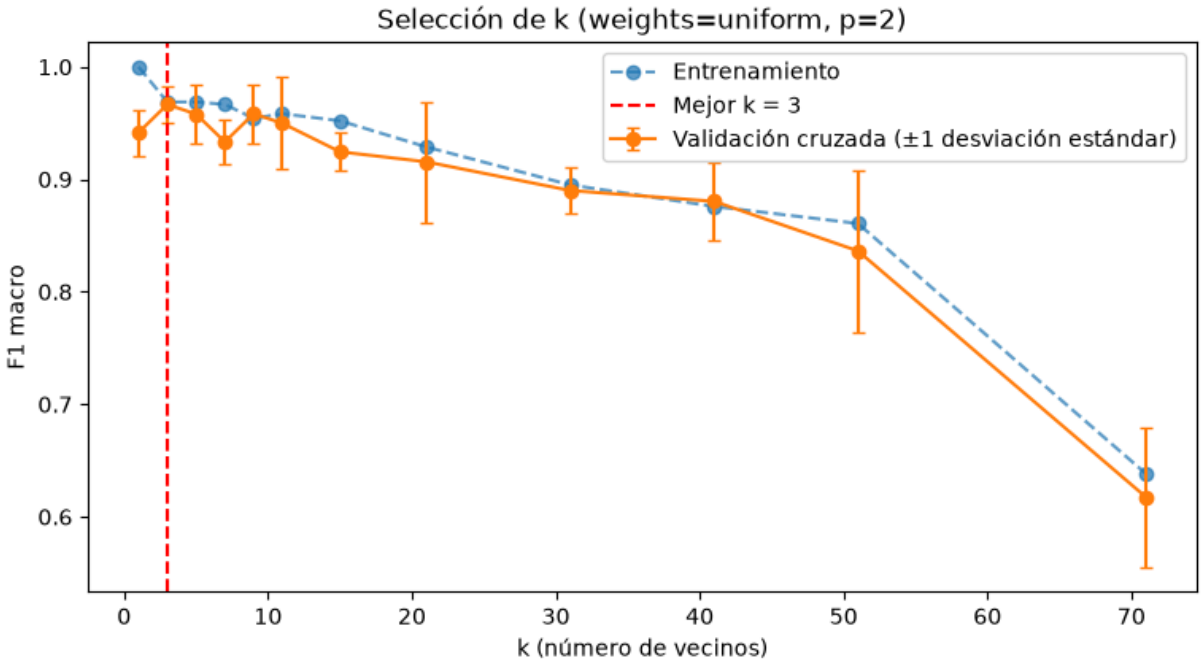
k_1se = int(eligible_ks.max())

plt.figure(figsize=(9, 4.5))
plt.plot(
    ks,
    curve["mean_train_score"],
    "o--",
    label="Entrenamiento",
    alpha=0.7
)
plt.errorbar(
    ks,
    curve["mean_test_score"],
    yerr=curve["std_test_score"],
    fmt="o-",
    capsize=3,
    label="Validación cruzada (±1 desviación estándar)"
```

```
)
plt.axvline(best_k, color="red", linestyle="--", label=f"Mejor k = {best_k}")
plt.xlabel("k (número de vecinos)")
plt.ylabel("F1 macro")
plt.title(f"Selección de k (weights={best_weights}, p={best_p})")
plt.legend()
plt.show()

print(
    f"Mejor k por media de CV: {best_k} "
    f"(F1 macro = {best_mean:.4f} ± {best_std:.4f})"
)
print(f"k elegido con la regla de un error estándar: {k_1se}")

print("\nTop 5 combinaciones:")
display(
    res.sort_values("rank_test_score")[
        [
            "param_knn__n_neighbors",
            "param_knn__weights",
            "param_knn__p",
            "mean_test_score",
            "std_test_score"
        ]
    ].head(5).round(4)
)
```




```
Mejor k por media de CV: 3 (F1 macro = 0.9665 ± 0.0167)
k elegido con la regla de un error estándar: 3

Top 5 combinaciones:
```

	param_knn__n_neighbors	param_knn__weights	param_knn__p	mean_test_score	std_test_s
6	3	uniform	2	0.9665	0.(
7	3	distance	2	0.9665	0.(
9	5	distance	1	0.9665	0.(
23	11	distance	2	0.9665	0.(
12	7	uniform	1	0.9582	0.(

Justificación de la elección de k

La búsqueda con validación cruzada de 5 folds seleccionó k = 3, con weights="uniform" y distancia euclidiana (p=2). El F1 macro promedio fue 0.9665, con desviación estándar entre folds de 0.0167.

Selección sin usar el conjunto de prueba: los hiperparámetros se buscaron usando únicamente X_train y y_train. El conjunto de prueba queda reservado para evaluar el modelo final.

Rendimiento y complejidad: en la curva se comparan los resultados de entrenamiento y validación cruzada para distintos valores de k. Un k pequeño puede ajustarse demasiado a los datos de entrenamiento; uno grande suaviza más la decisión y puede perder capacidad para distinguir clases. En esta ejecución, k = 3 obtuvo el mayor F1 macro promedio de validación. Variabilidad entre folds: la desviación estándar de 0.0167 muestra cuánto varió el F1 macro entre los folds. La regla de un error estándar también seleccionó k = 3; por tanto, con este criterio no se eligió una alternativa más grande.

Rango explorado: se evaluaron valores de k desde 1 hasta 71. El mejor valor, 3, no está en un extremo del rango, así que no hay indicios en esta búsqueda de que sea necesario ampliarlo hacia valores menores o mayores. Conclusión: se conserva k = 3 porque obtuvo el mejor F1 macro promedio en la validación cruzada y también fue seleccionado por la regla de un error estándar. La evaluación del conjunto de prueba debe reportarse por separado como estimación final del rendimiento.

3. **Present the dataset and process (30 minutes).** Explain decisions and results, and discuss possible improvements.

Classification metrics (30 %) – evaluación final en test

Ahora, **por primera y única vez**, se evalúa el modelo elegido en el conjunto de test (datos que nunca intervinieron en ninguna decisión). `GridSearchCV` ya reentrenó el mejor modelo con todo train (`refit=True`)

In [17]:
```python
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    f1_score,
    classification_report,
    ConfusionMatrixDisplay
)
import matplotlib.pyplot as plt

# Recuperar el mejor modelo ya reentrenado con todo X_train.
modelo_final = grid.best_estimator_

print("Hiperparámetros elegidos:", grid.best_params_)

# Evaluación final: predecir sobre el conjunto de prueba una sola vez.
predicciones_test = modelo_final.predict(X_test)

print(f"\nAccuracy en test: {accuracy_score(y_test, predicciones_test):.4f}")
print(
    "Balanced accuracy en test: "
    f"{balanced_accuracy_score(y_test, predicciones_test):.4f}"
)
print(f"F1 macro en test: {f1_score(y_test, predicciones_test, average='macro'):.4f

print("\nReporte de clasificación:")
print(classification_report(y_test, predicciones_test, zero_division=0))

ConfusionMatrixDisplay.from_predictions(
    y_test,
    predicciones_test,
    cmap="Blues",
    values_format="d",
    xticks_rotation="vertical"
)
plt.title("Matriz de confusión — conjunto de test")
plt.tight_layout()
plt.show()
```

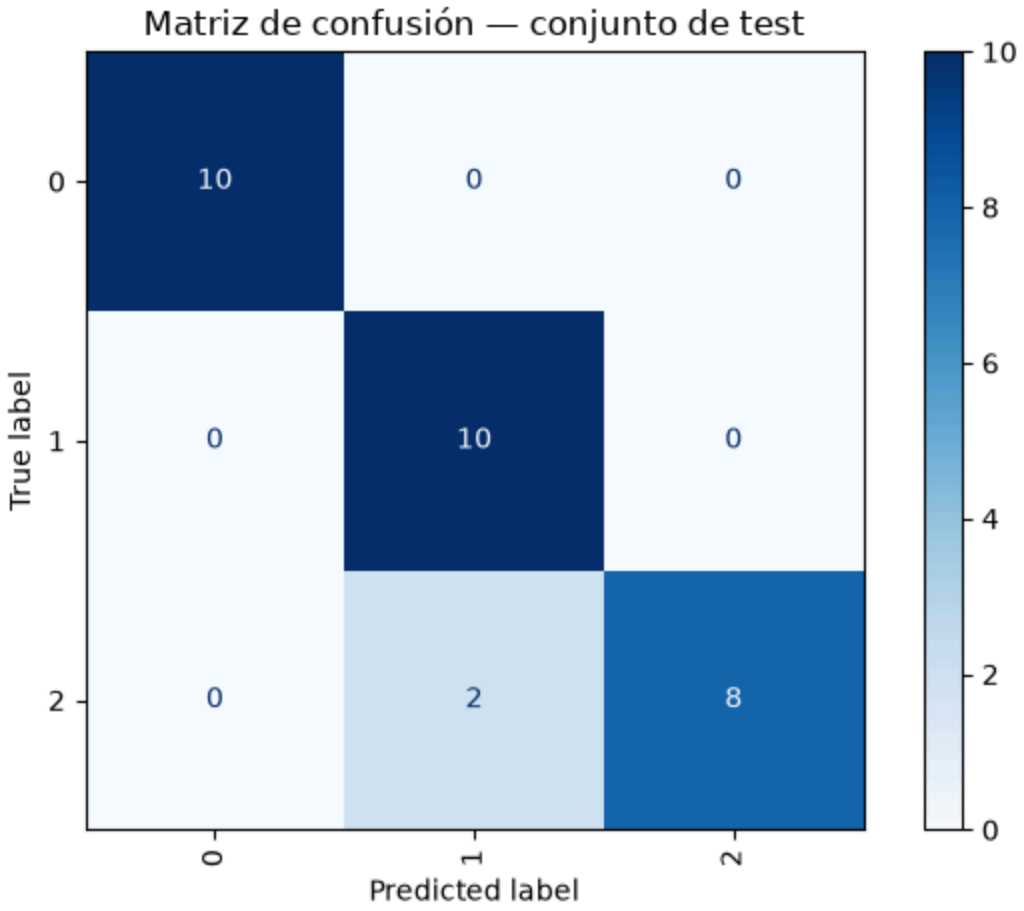
```
Hiperparámetros elegidos: {'knn__n_neighbors': 3, 'knn__p': 2, 'knn__weights': 'unif
orm'}

Accuracy en test: 0.9333
Balanced accuracy en test: 0.9333
F1 macro en test: 0.9327

Reporte de clasificación:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       0.83      1.00      0.91        10
           2       1.00      0.80      0.89        10

    accuracy                           0.93        30
   macro avg       0.94      0.93      0.93        30
weighted avg       0.94      0.93      0.93        30
```



Assessment

- Dataset source: 10%.
- Training/test split: 20%.
- Justification of the chosen `k`: 40%.
- Classification metrics: 30%.